

Celestial Gatekeeper Official Writeup

22nd October 2025

Prepared By: Hybread

Challenge Author: Hybread

Difficulty: Medium/Hard

Writeup classification: Official

1 – Introduction

Celestial Gatekeeper is a hard reversing challenge which involves a 32-bit crackme executable which implements x64-bit code utilizing the [Heaven's Gate technique \(WoW64\)](#). The user must go through the weird looking decompiled code, identify the Heaven's Gate calls, which they can then find each function's real instructions dynamically.

2 – Description

Have you ever seen or heard of a biblically accurate angel? They say it can take up multiple forms, sometimes appearing large, small, unique or even terrifying than what we're used to hearing...

3 – Skills Required

- Intermediate reverse engineering knowledge
- Well-versed in assembly
- Good knowledge in static/dynamic decompiling
- x86 architectural knowledge on segmentation registers

4 – Skills Learnt

- Bypassing basic anti-debugging/anti-decompiling techniques
- Learn about Heaven's Gate technique and CPU mode switching
- Learn how to bypass Heaven's Gate x32/x64 polyglot code with dynamic debugging

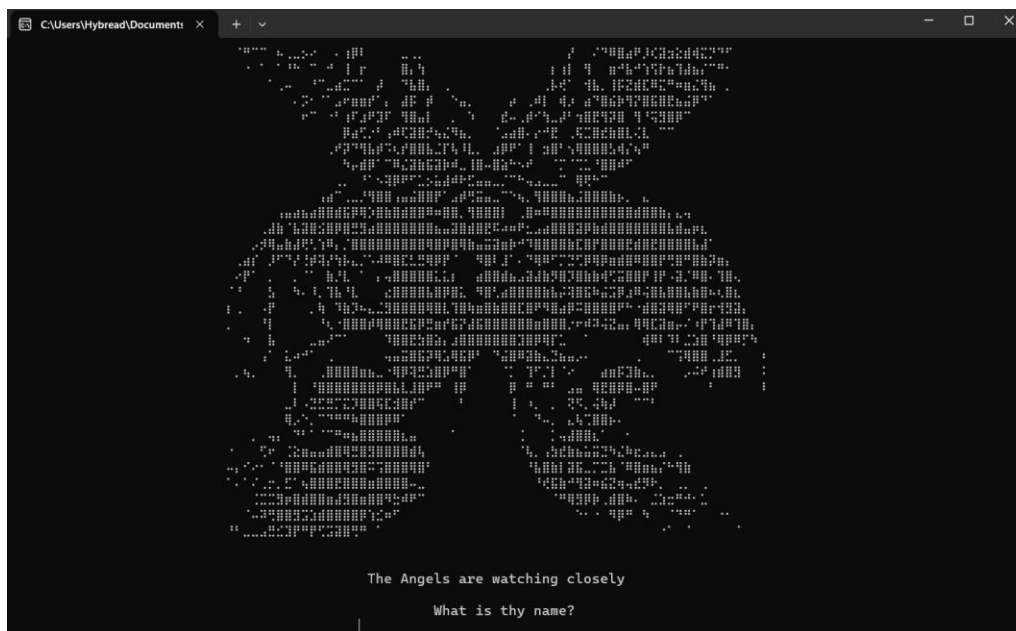
5 – Solve

Download the given .exe named 'gate.exe'.

If we run the file command on executable, we can see that it's an Windows x86 (32-bit) executable file:

```
(kali@kali)-[~/Desktop]
$ file gate.exe
gate.exe: PE32 executable for MS Windows 6.00 (console), Intel i386, 5 sections
```

Running the executable opens the terminal which displays a standard 'prompting' program:



The program seems to ask for a name? And after we input a dummy input, we get an error message:

```
The Angels are watching closely

What is thy name?
adssd

The Angels pause...
Such a pitifully brief name.. The Angels weep at this lack of grandeur!
```

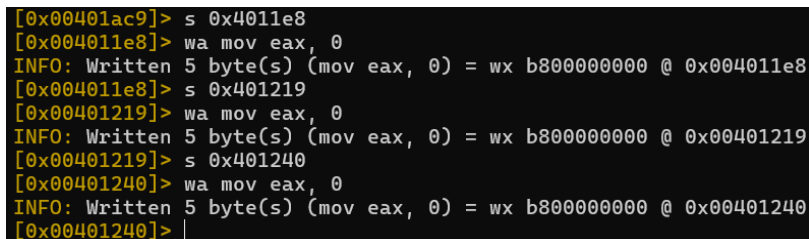
Now we know that it behaves similarly to a standard 'crackme' program.

We'll hop on IDA for static analysis and radare2 for dynamic debugging. Let's see if we can dig anything up on IDA first:



```
44  else
45  {
46      v24 = 0;
47      CurrentProcess = GetCurrentProcess();
48      CheckRemoteDebuggerPresent(CurrentProcess, (PB00L)&v24);
49      if ( v24 )
50      {
51          v4 = 1;
52      }
53      else
54      {
55          SetLastError(0);
56          OutputDebugStringA("dbgcheck");
57          if ( GetLastError() )
58          {
59              v4 = 1;
60          }
61          else
62          {
63              QueryPerformanceFrequency(&v20);
64              QueryPerformanceCounter(&v21);
65              Sleep(0);
66              QueryPerformanceCounter(&v23);
67              v4 = (double)(v23.QuadPart - v21.QuadPart) * 1000.0 / (double)v20.Qu
68          }
69      }
70  }
71  if ( v4 )
72      ExitProcess(1u);
```

As soon as we enter the main function, we're able to see a very basic anti debugging function. We can bypass this by simple patching each 'eax' value checks from 1 to 0 in r2 as such:



```
[0x004011e8]> s 0x4011e8
[0x004011e8]> wa mov eax, 0
INFO: Written 5 byte(s) (mov eax, 0) = wx b800000000 @ 0x004011e8
[0x004011e8]> s 0x401219
[0x00401219]> wa mov eax, 0
INFO: Written 5 byte(s) (mov eax, 0) = wx b800000000 @ 0x00401219
[0x00401219]> s 0x401240
[0x00401240]> wa mov eax, 0
INFO: Written 5 byte(s) (mov eax, 0) = wx b800000000 @ 0x00401240
[0x00401240]> |
```

Save save the patched version, and we go back to IDA to confirm the patched instruction:

```

.text:005F11E0      jz         short loc_5F11F2
.text:005F11E8      mov        eax, 0
.text:005F11ED      jmp        loc_5F12CA
.text:005F11F2      ; -----
.text:005F11F2      loc_5F11F2:      ; CODE XREF: _main+36↑j
.text:005F11F2      lea        eax, [ebp-200h]
.text:005F11F8      mov        dword ptr [ebp-200h], 0
.text:005F1202      push      eax      ; pbDebuggerPresent
.text:005F1203      call      ds:GetCurrentProcess
.text:005F1209      push      eax      ; hProcess
.text:005F120A      call      ds:CheckRemoteDebuggerPresent
.text:005F1210      cmp        dword ptr [ebp-200h], 0
.text:005F1217      jz         short loc_5F1223
.text:005F1219      mov        eax, 0
.text:005F121E      jmp        loc_5F12CA
.text:005F1223      ; -----
.text:005F1223      loc_5F1223:      ; CODE XREF: _main+67↑j
.text:005F1223      push      0      ; dwErrCode
.text:005F1225      call      ds:SetLastError
.text:005F122B      push      offset OutputString ; "dbgcheck"
.text:005F1230      call      ds:OutputDebugStringA
.text:005F1236      call      ds:GetLastError
.text:005F123C      test      eax, eax
.text:005F123E      jz         short loc_5F124A
.text:005F1240      mov        eax, 0

```

Now we confirmed that the debugger check is gone, we can continue with our static analysis before going into dynamic debugging.

The first part of the main function of the program's code looks normal:

<pre> B61130 var_28 = dword ptr -28h B61130 B61130 push ebp B61131 mov ebp, esp B61133 push ecx B61134 push ebx B61135 push esi B61136 push edi B61137 push ebx B61138 sub esp, 0Ch B6113B mov ebx, offset unk_B7F090 B61140 push 33h ; '3' B61142 call \$+5 B61147 add [esp+28h+var_28], 5 B61148 retf B61148 sub_B61130 endp ; sp-analysis failed B61148 B6114C ; ----- B6114C dec eax B6114D mov eax, [ebx] B6114F dec ecx B61150 mov eax, eax B61152 dec ecx B61153 shr eax, 3Fh B61156 dec ecx B61157 mov ecx, eax B61159 dec ecx B6115A shr ecx, 3Eh B6115D dec ebp </pre>	<pre> 1 void sub_B61130() 2 { 3 JUMPOUT(0xB7260C); 4 } </pre>
--	---

But it soon becomes weird. Here if we look closely, on the first real function call, we see that IDA is bugging out with a failed analysis and followed by a bunch of nonsense instructions. What's going on?

Let's analyze in orderly fashion starting by going back to the main function:

First, we can see that the program checks for the user input:

```
...  
  
if ( &v27[strlen(&v26)] - v27 == 34 )  
{  
    for ( i = 0; i < 0x22; ++i )  
        *((_BYTE *)&v28 + i) = (2 * v27[i - 1]) ^ ((2 * v27[i - 1]) ^ ((unsigned __int8)v27[i -  
1] >> 1)) & 0x55;  
...  
}
```

We see that this loop checks whether the user input is exactly 34 bytes long meaning we'll need to look out for a string of 34 characters. After that, it performs some sort of mathematical operation carried out and by simply looking at it, we can derive that it's a bit-twiddling operation which looks a little something like this:

```
...  
  
(2 * x) ^ ((2 * x) ^ (x >> 1)) & 0x55  
  
...
```

We can rewrite it in high-level C like this:

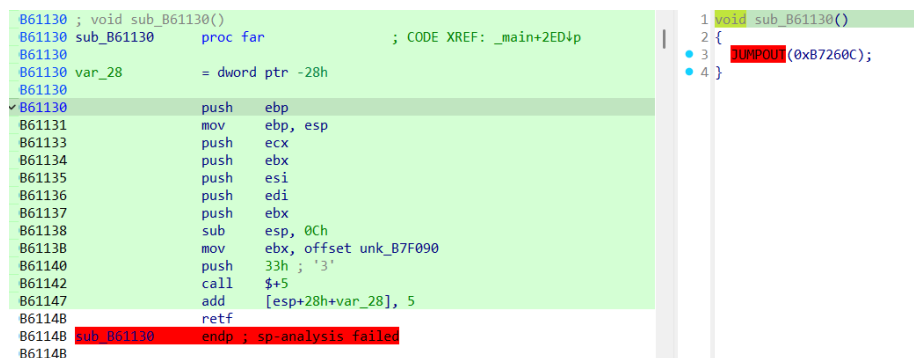
```
static uint8_t bitTwiddle(uint8_t x) {  
    uint8_t a = (uint8_t)(2u * (uint32_t)x);  
    uint8_t b = (uint8_t)(a ^ (x >> 1));  
    return (uint8_t)(a ^ (b & 0x55u));  
}
```

```

104     v13 = 33;
105     v33 = v30;
106     v31 = v28;
107     v32 = v29;
108     v22.HighPart = 34 - ((_DWORD)&v33 + 1);
109     do
110     {
111         v14 = (char *)&v31 + v13;
112         v24 = *((_BYTE *)&v31 + v13);
113         sub_B61130();
114         v16 = v15 % ((int)&v31 + v13 + v22.HighPart);
115         --v13;
116         *v14 = *((_BYTE *)&v31 + (unsigned __int8)v16);
117         *((_BYTE *)&v31 + (unsigned __int8)v16) = v24;
118     }
119     while ( v13 >= 0 );

```

Right after that, we have this algorithm that's implemented after the bit twiddling. What this does is first declaring the loop index to start on the 33rd element defined by `v13 = 33;`, then calling the function we saw earlier `sub\_B61130` to perform some sort of instruction but what does it do? Let's inspect it:



The screenshot shows the disassembly and decompiled code for the function `sub\_B61130`.

Disassembly View (Left):

```

B61130 ; void sub_B61130()
B61130 sub_B61130      proc far          ; CODE XREF: _main+2ED4p
B61130
B61130 var_28          = dword ptr -28h
B61130
B61130 ✓ B61130          push     ebp
B61131                  mov     ebp, esp
B61133                  push     ecx
B61134                  push     ebx
B61135                  push     esi
B61136                  push     edi
B61137                  push     ebx
B61138                  sub     esp, 0Ch
B61138                  mov     ebx, offset unk_B7F090
B61140                  push     33h ; '3'
B61142                  call    $+5
B61147                  add     [esp+28h+var_28], 5
B61148                  retf
B61148 sub_B61130      endp ; sp-analysis failed
B61148

```

Decompiler View (Right):

```

1 void sub_B61130()
2 {
3     JUNKPOP(0xB7260C);
4 }

```

We can clearly see that `33h` or `0x33` is being pushed into the stack, calls the next instruction, then it adjusts the return address on the stack to skip 5 bytes of call instruction when returning to setup the `retf` call. [The far return then pops both the return address and the segment selector \(0x33\) from the stack, loading CS with 0x33 which switches the CPU to 64-bit mode.](#) This helps us identify that we're dealing with a [heaven's gate transition](#).

So what's going on in the 64-bit code? Well there's many ways to view it, but we'll be doing this in our trusty Radare2. To do so we'll simply load the patched executable with r2 with the following command: `r2 -AAA gate.exe`

We'll first seek out the address of the segment where this function is called based on our IDA which will be at `0xB91490`

```
.text:00B91490 loc_B91490: ; CODE XREF: _main+316+j
.text:00B91490 lea esi, [ebp-40h]
.text:00B91493 add esi, edi
.text:00B91495 mov al, [esi]
.text:00B91497 mov [ebp-1F9h], al
.text:00B914A0 call near ptr sub_B91130
.text:00B914A2 mov ecx, [ebp-204h]
.text:00B914A8 movzx eax, al
.text:00B914AB add ecx, esi
.text:00B914AD cdq ecx
.text:00B914AE idiv ecx
.text:00B914B0 sub edi, 1
.text:00B914B3 movzx ecx, dl
.text:00B914B6 mov al, [ebp+ecx-40h]
.text:00B914BA mov [esi], al
.text:00B914BC mov al, [ebp-1F9h]
.text:00B914C2 mov [ebp+ecx-40h], al
.text:00B914C6 jns short loc_B91490
```

```
108 v33 = v30;
109 v23.HighPart = 34 - ((DWORD)&v34 + 1);
110 do
111 {
112     v14 = (char *)&v32 + v13;
113     v25 = *((_BYTE *)&v32 + v13);
114     v15 = sub_B91130(v19);
115     v16 = v15 % ((int)&v32 + v13 + v23.HighPart);
116     --v13;
117     *v14 = *((_BYTE *)&v32 + (unsigned __int8)v16);
118     *((_BYTE *)&v32 + (unsigned __int8)v16) = v25;
119 }
120 while ( v13 >= 0 );
121 v17 = 0;
122 v29 = v32;
123 v30 = v33;
124 v31 = v34;
125 do
```

```
[0x00b91490]> pd 100
; CODE XREF from main @ 0xb914c6(x)
-> 0x00b91490 8d75c0 lea esi, [ebp - 0x40]
0x00b91493 03f7 add esi, edi
0x00b91495 8a06 mov al, byte [esi]
0x00b91497 88507feffff mov byte [ebp - 0x1f9], al
0x00b9149d e88efcffff call 0xb91130
0x00b914a2 8b8dfcfdffff mov ecx, dword [ebp - 0x204]
0x00b914a8 0fb6c0 movzx eax, al
0x00b914ab 03ce add ecx, esi
0x00b914ad 99 cdq
0x00b914ae f7f9 idiv ecx
0x00b914b0 83ef01 sub edi, 1
0x00b914b3 0fb6ca movzx ecx, dl
0x00b914b6 8a440dc0 mov al, byte [ebp + ecx - 0x40]
0x00b914ba 8806 mov byte [esi], al
0x00b914bc 8a507feffff mov al, byte [ebp - 0x1f9]
0x00b914c2 88440dc0 mov byte [ebp + ecx - 0x40], al
0x00b914c6 79c8 jns 0xb91490
0x00b914c8 0f2845c0 movaps xmm0, xmmword [ebp - 0x40]
0x00b914cc 33f6 xor esi, esi
0x00b914ce 668b45e0 mov ax, word [ebp - 0x20]
```

Following the program flow, we see a call to `0xB91130` which matches the IDA function call. Let's follow the call to the function and see what we're dealing with:

```
[0x00b61490]> s 0xB61130
[0x00b61130]> pd 50
; CALL XREF from sub.main_4011b0 @ 0xb6149d(x)
/ 23: sub.a0___401130 ();
0x00b61130 55 push ebp
0x00b61131 8bec mov ebp, esp
0x00b61133 51 push ecx
0x00b61134 53 push ebx
0x00b61135 56 push esi
0x00b61136 57 push edi
0x00b61137 53 push ebx
0x00b61138 83ec0c sub esp, 0xc
0x00b6113b bb90f0b700 mov ebx, str.a0___ ; 0xb7f090 ;
0x00b61140 6a33 push 0x33 ; '3' ; 51
0x00b61142 e800000000 call fcn.00401147
; CALL XREF from sub.a0___401130 @ 0xb61142(x)
/ 5: fcn.00401147 ();
0x00b61147 83042405 add dword [esp], 5
0x00b6114b cb retf
0x00b6114c 48 dec eax
0x00b6114d 8b03 mov eax, dword [ebx]
0x00b6114f 49 dec ecx
0x00b61150 89c0 mov eax, eax
0x00b61152 49 dec ecx
0x00b61153 c1e83f shr eax, 0x3f
0x00b61156 49 dec ecx
0x00b61157 89c1 mov ecx, eax
0x00b61159 49 dec ecx
0x00b6115a c1e93e shr ecx, 0x3e
```

After seeking the address of the function, we dump 50 lines of instructions and we're able to see the heaven's gate transition. Again, due to the executable being loaded in 32-bit

initially, we're seeing a bunch of instructions that doesn't make sense. Let's change it to 64-bit mode to see the real instructions:

```
[0x00b91130]> e asm.bits = 64
[0x00b91130]> pd 50
; CALL XREF from main @ 0xb9149d(x)
0x00b91130      55      push rbp
0x00b91131      8bec     mov ebp, esp
0x00b91133      51      push rcx
0x00b91134      53      push rbx
0x00b91135      56      push rsi
0x00b91136      57      push rdi
0x00b91137      53      push rbx
0x00b91138      83ec0c   sub esp, 0xc
0x00b9113b      bb90f0ba00 mov ebx, str.a0__
0x00b91140      6a33     push 0x33
0x00b91142      e800000000 call 0xb91147
; CALL XREF from section..text @ +0x142(x)
0x00b91147      83042405 add dword [rsp], 5
0x00b9114b      cb       retf
0x00b9114c      488b03   mov rax, qword [rbx]
0x00b9114f      4989c0   mov r8, rax
0x00b91152      49c1e83f shr r8, 0x3f
0x00b91156      4989c1   mov r9, rax
0x00b91159      49c1e93e shr r9, 0x3e
0x00b9115d      4d31c8   xor r8, r9
0x00b91160      4989c1   mov r9, rax
0x00b91163      49c1e93c shr r9, 0x3c
0x00b91167      4d31c8   xor r8, r9
0x00b9116a      4989c1   mov r9, rax
0x00b9116d      49c1e93b shr r9, 0x3b
0x00b91171      4d31c8   xor r8, r9
0x00b91174      4983e001 and r8, 1
0x00b91178      48d1e0   shl rax, 1
```

Now we have the actual instructions and we're able to interpret what's going on.

This following function is essentially a cross-mode LFSR (Linear-feedback Shift Register) where the instruction `mov rax, [rbx]` loads a 64-bit LFSR state shown on the right from memory at address `0xBAF090` into rax register. Then, it shifts the state right to extract bits 63, 62, 60, 59, XORs them together, and masks to 1 bit. The state is then shifted again to the left by one bit through `shl rax, 1`, and the feedback bit is inserted at the least significant bit and then writing the new state back to the rbx register. To reverse this function, we'll be required to hunt down the value of the LFSR state's value as shown to be a `str`. To do this we just need to dump the value from the address, and we can do that by using this command `pxq @ 0xbaf090`, and we get our state's seed:

```
[0x7797b6f0]> pxq @ 0xbaf090
0x00baf090  0x7f3a1c9e5b2d4f61  0x0000000000000000  a0-[...:.....
0x00baf0a0  0x0000000000000000  0x0000000000000000  .....
0x00baf0b0  0x0000000000000000  0x0000000000000000  .....
```

And there we have the 64-bit integer of `0x7F3A1C9E5B2D4F61` which acts as the state seed for the LFSR which maintains the pseudo-random index generation.

Now that we have the full LFSR function, we can interpret it in high-level C code such as the following:

```
uint8_t lfsr() {  
    static uint64_t state = 0x7F3A1C9E5B2D4F61ULL;  
    uint64_t fb = ((state >> 63) ^ (state >> 62) ^ (state >> 60) ^ (state >> 59)) & 1;  
    state = (state << 1) | fb;  
    return (uint8_t)state;  
}
```

Now we know that function `sub\_B91130` is a pseudo-random seed generator, we can map out the entire mathematical operation carried out by the program:

...

v13 = 33; = The loop index starts at 33 (last element in the username array)

v15 = sub_601130(v19); = LFSR seed generation

v16 = v15 % ((int)&v32 + v13 + v23.HighPart); = The result of the seed is taken mod (i+1).
Which selects a random index from 0 to i.

--v13; = The loop counts downward from 33 to 0, a Fisher-Yates shuffling algorithm

*v14 = *((_BYTE *)&v32 + v16); = Swap step: moves one element.

*((_BYTE *)&v32 + v16) = v25; = Completes the swap of bytes i and j.

while (v13 >= 0); = Loop until all positions are shuffled.

...

And now we can replicate the function in high-level C as such:

```
158 for (int i = 33; i >= 0; --i) {  
159     uint8_t r = lfsr(); // sub_601130  
160     int j = r % (i + 1);  
161     uint8_t tmp = arr[i];  
162     arr[i] = arr[j];  
163     arr[j] = tmp;  
164 }
```

Going back to IDA, we see the next function call to `sub\_B910A0` and clicking into it also shows the same JUMPOUT error. We're also able to see another heaven's gate transition for that specific function:

<pre> .text:00B910A0 ; Attributes: bp-based frame .text:00B910A0 .text:00B910A0 ; void sub_B910A0() .text:00B910A0 sub_B910A0 proc far ; CODE XREF: _main+3384p .text:00B910A0 .text:00B910A0 var_20 = dword ptr -20h .text:00B910A0 var_4 = dword ptr -4 .text:00B910A0 ✓.text:00B910A0 push ebp .text:00B910A1 mov ebp, esp .text:00B910A3 push ecx .text:00B910A4 movzx eax, byte ptr [ebp+8] .text:00B910A8 push ebx .text:00B910A9 push esi .text:00B910AA push edi .text:00B910AB mov [ebp+var_4], 0 .text:00B910B2 mov [ebp+8], eax .text:00B910B5 push ebx .text:00B910B6 push esi .text:00B910B7 mov esi, [ebp+8] .text:00B910BA lea ebx, [ebp+var_4] .text:00B910BD push 33h ; '3' .text:00B910BF call \$+5 .text:00B910C4 add [esp+20h+var_20], 5 .text:00B910C8 retf .text:00B910C8 sub_B910A0 endp ; sp-analysis failed +avt+00B910C8 </pre>	<pre> 1 void sub_B910A0() 2 { 3 JUMPOUT(0xBA1D59); 4 } </pre>
--	---

But wait, in the segment itself, after the function is called, there's ANOTHER heaven's gate transition hidden in plain sight.

<pre> .text:00B914E2 loc_B914E2: movzx eax, byte ptr [ebp+esi-70h] ; CODE XREF: _main+34B4j .text:00B914E2 .text:00B914E7 push eax .text:00B914E8 call near ptr sub_B910A0 .text:00B914ED mov [ebp+esi*4-1F8h], eax .text:00B914F4 add esp, 4 .text:00B914F7 inc esi .text:00B914F8 cmp esi, 22h ; '''' .text:00B914FB jb short loc_B914E2 .text:00B914FD push 1 ; Ix .text:00B914FF call __acrt_iob_func ; Ix .text:00B91504 push eax ; Stream .text:00B91505 call _fflush .text:00B9150A add esp, 8 .text:00B9150D push ebx .text:00B9150E lea ecx, [ebp-1F8h] .text:00B91514 lea edx, unk_BAF008 .text:00B9151A lea ebx, [ebp-20Ch] .text:00B91520 push 33h ; '3' .text:00B91522 call \$+5 .text:00B91527 add [esp+24Ch+var_24C], 5 .text:00B91528 retf </pre>	<pre> 109 v23.HighPart = 34 - ((DWORD)&v34 + 1); 110 do 111 { 112 v14 = (char *)&v32 + v13; 113 v25 = *((_BYTE *)&v32 + v13); 114 v15 = sub_B91130(v19); 115 v16 = v15 % ((int)&v32 + v13 + v23.HighPart); 116 --v13; 117 *v14 = *((_BYTE *)&v32 + (unsigned __int8)v16); 118 *((_BYTE *)&v32 + (unsigned __int8)v16) = v25; 119 } 120 while (v13 >= 0); 121 v17 = 0; 122 v29 = v32; 123 v30 = v33; 124 v31 = v34; 125 do 126 { 127 v26[v17] = sub_B910A0(*((unsigned __int8 *)&v29 + v17)); 128 ++v17; 129 } 130 while (v17 < 0x22); 131 v18 = __acrt_iob_func(1u); </pre>
--	---

Let's analyze this in r2 and see what's going on in this segment:

```

[0x00b91130]> s 0xb914E2
[0x00b914e2]> pd 50
: ; CODE XREF from main @ 0xb914fb(x)
--> 0x00b914e2 0fb6443590 movzx eax, byte [ebp + esi - 0x70]
: 0x00b914e7 50 push eax
: 0x00b914e8 e8b3fbffff call 0xb910a0
: 0x00b914ed 8984b508fe.. mov dword [ebp + esi*4 - 0x1f8], eax
: 0x00b914f4 83c404 add esp, 4
: 0x00b914f7 46 inc esi
: 0x00b914f8 83fe22 cmp esi, 0x22 ; '"' ; 34
==< 0x00b914fb 72e5 jnb 0xb914e2
: 0x00b914fd 6a01 push 1 ; 1
: 0x00b914ff e844310000 call 0xb94648
: 0x00b91504 50 push eax
: 0x00b91505 e8b9350000 call 0xb94ac3
: 0x00b9150a 83c408 add esp, 8
: 0x00b9150d 53 push ebx
: 0x00b9150e 8d8d08feffff lea ecx, [ebp - 0x1f8]
: 0x00b91514 8d1508f0ba00 lea edx, [0xbaf000]
: 0x00b9151a 8d9df4fdffff lea ebx, [ebp - 0x20c]
: 0x00b91520 6a33 push 0x33 ; '3' ; 51
: 0x00b91522 e800000000 call 0xb91527 ; int main(int argc)
: ; CALL XREF from main @ 0xb91522(x)
: 0x00b91527 83042405 add dword [esp], 5
: 0x00b9152b cb retf
: 0x00b9152c 48 dec eax
: 0x00b9152d 89cf mov edi, ecx
: 0x00b9152f 48 dec eax
: 0x00b91530 89d6 mov esi, edx
: 0x00b91532 fc cld
: 0x00b91533 b964000000 mov ecx, 0x64 ; 'd' ; 100

```

Seeking the address of the segment `0xB914E2`, we're able to see the same function call of `sub\_B910A0` in IDA, followed by a heaven's gate transition. We'll investigate this transition after we analyze the function `0xb910a0` first:

```

[0x00b914e2]> s 0xb910a0
[0x00b910a0]> pd 50
: ; CALL XREF from main @ 0xb914e8(x)
: 0x00b910a0 55 push ebp
: 0x00b910a1 8bec mov ebp, esp
: 0x00b910a3 51 push ecx
: 0x00b910a4 0fb64508 movzx eax, byte [ebp + 8]
: 0x00b910a8 53 push ebx
: 0x00b910a9 56 push esi
: 0x00b910aa 57 push edi
: 0x00b910ab c745fc0000.. mov dword [ebp - 4], 0
: 0x00b910b2 894508 mov dword [ebp + 8], eax
: 0x00b910b5 53 push ebx
: 0x00b910b6 56 push esi
: 0x00b910b7 8b7508 mov esi, dword [ebp + 8]
: 0x00b910ba 8d5dfc lea ebx, [ebp - 4]
: 0x00b910bd 6a33 push 0x33 ; '3' ; 51
: 0x00b910bf e800000000 call 0xb910c4
: ; CALL XREF from section..text @ +0xbf(x)
: 0x00b910c4 83042405 add dword [esp], 5
: 0x00b910c8 cb retf
: 0x00b910c9 48 dec eax
: 0x00b910ca 89df mov edi, ebx
: 0x00b910cc b8efbecaca mov eax, 0xcacabeef
: 0x00b910d1 89f1 mov ecx, esi
: 0x00b910d3 89ca mov edx, ecx

```

As seen in IDA, it's another heaven's gate transition but we can see a key of some sort with the value `0xcacabeef` being loaded in the eax register in the beginning. Let's expand it to see the full function:

```

0x00b910c4 83042405 add dword [esp], 5
0x00b910c8 cb retf
0x00b910c9 48 dec eax
0x00b910ca 89df mov edi, ebx
0x00b910cc b8efbecaca mov eax, 0xcacabeef
0x00b910d1 89f1 mov ecx, esi
0x00b910d3 89ca mov edx, ecx
0x00b910d5 c1e218 shl edx, 0x18
0x00b910d8 31d0 xor eax, edx
0x00b910da 89c2 mov edx, eax
0x00b910dc c1ea07 shr edx, 7
0x00b910df 89c3 mov ebx, eax
0x00b910e1 c1e305 shl ebx, 5
0x00b910e4 31da xor edx, ebx
0x00b910e6 31d0 xor eax, edx
0x00b910e8 89c2 mov edx, eax
0x00b910ea c1e20d shl edx, 0xd
0x00b910ed 89c3 mov ebx, eax
0x00b910ef c1eb0b shr ebx, 0xb
0x00b910f2 31da xor edx, ebx
0x00b910f4 31d0 xor eax, edx
0x00b910f6 89c2 mov edx, eax
0x00b910f8 c1ea11 shr edx, 0x11
0x00b910fb 69c0feca3713 imul eax, eax, 0x1337cafe
0x00b91101 31d0 xor eax, edx
0x00b91103 89c2 mov edx, eax
0x00b91105 c1e203 shl edx, 3
0x00b91108 01ca add edx, ecx
0x00b9110a 31d0 xor eax, edx
0x00b9110c 8907 mov dword [edi], eax

```

From what we can analyze from the instructions, it's a custom hashing and xor-shifting function which `uint32_t` seeds ``0xCACABEEF`` and ``0x1337CAFE``, then carries out xors to diffuse bits of the user input.

So mathematically, it should look like this:

```

...

```

```

uint32_t h = 0xCACABEEF;
h ^= (c << 24);
h ^= ((h >> 7) ^ (h << 5));
h ^= ((h << 13) ^ (h >> 11));
h = (h * 0x1337CAFE) ^ (h >> 17);
h ^= ((h << 3) + c);
*[edi] = h;

```

```

...

```

And converting it to high-level C would look like this:

```
static uint32_t map_byte(uint8_t input) {
    uint32_t eax = 0xcacabeefu;
    uint32_t ecx = (uint32_t)input;
    uint32_t edx, ebx;

    edx = ecx << 24;
    eax ^= edx;

    edx = eax >> 7;
    ebx = eax << 5;
    edx ^= ebx;
    eax ^= edx;

    edx = eax << 13;
    ebx = eax >> 11;
    edx ^= ebx;
    eax ^= edx;

    edx = eax >> 17;
    uint64_t tmp = (uint64_t)eax * 0x1337CAFEULL;
    eax = (uint32_t)tmp;
    eax ^= edx;

    edx = eax << 3;
    edx += ecx;
    eax ^= edx;

    return eax;
}
```

Now we know what this function does, we can go back and analyze the heaven's gate transition that's happening in the segment itself.

```
0x00b9150e 8d8d08feffff lea ecx, [ebp - 0x1f8]
0x00b91514 8d1508f0ba00 lea edx, [0xbaf008]
0x00b9151a 8d9df4fdffff lea ebx, [ebp - 0x20c]
0x00b91520 6a33          push 0x33          ; '3' ; 51
0x00b91522 e800000000    call 0xb91527      ; int main(int argc, c
; CALL XREF from main @ 0xb91522(x)
0x00b91527 83042405      add dword [esp], 5
0x00b9152b cb           retf
0x00b9152c 48           dec eax
0x00b9152d 89cf         mov edi, ecx
0x00b9152f 48           dec eax
0x00b91530 89d6         mov esi, edx
0x00b91532 fc           cld
0x00b91533 b964000000    mov ecx, 0x64      ; 'd' ; 100
0x00b91538 f3a6         repe cmpsb byte [esi], byte es:[edi]
0x00b9153a 7506         jne 0xb91542
0x00b9153c c70301000000 mov dword [ebx], 1
; CODE XREF from main @ +0x38a(x)
0x00b91542 e800000000    call 0xb91547
; CALL XREF from main @ +0x392(x)
0x00b91547 c744240423.. mov dword [esp + 4], 0x23 ; '#'
; [0x23:4]=-1 ; 35
0x00b9154f 8304240d      add dword [esp], 0xd ; [0xd:4]=-1 ; 13
0x00b91553 cb           retf
```

From what we can see, this gate essentially compares two arrays (expected and input) of 32-bit hashes using a byte-wise comparison and below we can see an array being loaded into the edx register at `0x00b91514`. Then compares 136 bytes of 32bit byte by byte (it shows 100 bytes now but I changed the program, just too lazy to fix this writeup). Similarly to how we dumped the values of the LFSR state seed, we can do the same with this array `0xbaf008`:

```

[0x00b914e2]> pxw 200 @ 0xbaf008
0x00baf008 0xc5665baf 0x4e6458b3 0x1146b41e 0x9931a0cf .[f..XdN..F...1.
0x00baf018 0x8a13b779 0xb4072c9b 0x8d938bb1 0xa8d6c3c7 y.....
0x00baf028 0x1146b41e 0x9d4b68c9 0xea172d72 0x965538a7 ..F..hK.r...8U.
0x00baf038 0x2b37a4a1 0x965538a7 0xbe9de767 0xc5665baf ..7+.8U.g....[f.
0x00baf048 0x58832448 0xe983dd00 0xb4072c9b 0x965538a7 H$.X.....8U.
0x00baf058 0xb96eab6d 0xb4072c9b 0x578b5381 0x1146b41e m.n.....S.W..F.
0x00baf068 0x13733bbb 0x1146b41e 0xe6d6d74d 0xf766b52c .;s...F.M....f.
0x00baf078 0x8a13b779 0x8a13b779 0x16d2c4b5 0xe6d6d74d y...y.....M...
0x00baf088 0xea172d72 0x1146b41e 0x5b2d4f61 0x7f3a1c9e r-....F.a0-[...:
0x00baf098 0x00000000 0x00000000 0x00000000 0x00000000 .....
0x00baf0a8 0x00000000 0x00000000 0x00000000 0x00000000 .....
0x00baf0b8 0x00000000 0x00000000 0xbb40e64e 0x00000000 .....N.@.....
0x00baf0c8 0x00000000 0x00000000 .....
[0x00b914e2]> pxw 136 @ 0xbaf008
0x00baf008 0xc5665baf 0x4e6458b3 0x1146b41e 0x9931a0cf .[f..XdN..F...1.
0x00baf018 0x8a13b779 0xb4072c9b 0x8d938bb1 0xa8d6c3c7 y.....
0x00baf028 0x1146b41e 0x9d4b68c9 0xea172d72 0x965538a7 ..F..hK.r...8U.
0x00baf038 0x2b37a4a1 0x965538a7 0xbe9de767 0xc5665baf ..7+.8U.g....[f.
0x00baf048 0x58832448 0xe983dd00 0xb4072c9b 0x965538a7 H$.X.....8U.
0x00baf058 0xb96eab6d 0xb4072c9b 0x578b5381 0x1146b41e m.n.....S.W..F.
0x00baf068 0x13733bbb 0x1146b41e 0xe6d6d74d 0xf766b52c .;s...F.M....f.
0x00baf078 0x8a13b779 0x8a13b779 0x16d2c4b5 0xe6d6d74d y...y.....M...
0x00baf088 0xea172d72 0x1146b41e r-....F.
[0x00b914e2]> |

```

We can see more than 34 chunks of `uint32_t` values dumping 200 with `pxw`, but since the program is only comparing the first 136 bytes, we can just take the first 34 chunks as our to-be computed values.

Now that we have everything, we can craft our full solve script in C:

```

...

```

```

#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <stdlib.h>

#define USERNAME_LEN 34
#define MASK64 ((uint64_t)0xFFFFFFFFFFFFFFFFULL)

// dumped expected array
static const uint32_t USERNAME[USERNAME_LEN] = {
    0xc5665baf, 0x4e6458b3, 0x1146b41e, 0x9931a0cf, 0x8a13b779,
    0xb4072c9b, 0x8d938bb1, 0xa8d6c3c7, 0x1146b41e, 0x9d4b68c9,
    0xea172d72, 0x965538a7, 0x2b37a4a1, 0x965538a7, 0xbe9de767,
    0xc5665baf, 0x58832448, 0xe983dd00, 0xb4072c9b, 0x965538a7,
    0xb96eab6d, 0xb4072c9b, 0x578b5381, 0x1146b41e, 0x13733bbb,
    0x1146b41e, 0xe6d6d74d, 0xf766b52c, 0x8a13b779, 0x8a13b779,
    0x16d2c4b5, 0xe6d6d74d, 0xea172d72, 0x1146b41e
};

static const uint64_t stateSeed = 0x7F3A1C9E5B2D4F61ULL;

```

```

// custom hash map
static uint32_t hash(uint8_t input) {
    uint32_t eax = 0xcacabeefu;
    uint32_t ecx = (uint32_t)input;
    uint32_t edx, ebx;

    /* h ^= (c << 24) */
    edx = ecx << 24;
    eax ^= edx;

    /* h ^= ((h >> 7) ^ (h << 5)) */
    edx = eax >> 7;
    ebx = eax << 5;
    edx ^= ebx;
    eax ^= edx;

    /* h ^= ((h << 13) ^ (h >> 11)) */
    edx = eax << 13;
    ebx = eax >> 11;
    edx ^= ebx;
    eax ^= edx;

    /* h = (h * 0x1337CAFE) ^ (h >> 17) */
    edx = eax >> 17;
    {
        uint64_t tmp = (uint64_t)eax * 0x1337CAFEULL;
        eax = (uint32_t)tmp;
    }
    eax ^= edx;

    /* h ^= ((h << 3) + c) */
    edx = (eax << 3) + ecx;
    eax ^= edx;

    return eax;
}

// per byte transformation (bit twiddling)
static uint8_t bitTwiddle(uint8_t x) {
    uint8_t a = (uint8_t)(2u * (uint32_t)x);
    uint8_t b = (uint8_t)(a ^ (x >> 1));
    return (uint8_t)(a ^ (b & 0x55u));
}

```

```

static uint8_t inversePerByte[256];
static int inverseTwiddle = 0;
static void inverseByte(void) {
    uint8_t seen[256] = {0};
    for (int x = 0; x < 256; ++x) {
        uint8_t y = bitTwiddle((uint8_t)x);
        inversePerByte[y] = (uint8_t)x;
        seen[y] = 1;
    }
    for (int y = 0; y < 256; ++y) {
        if (!seen[y]) {
            fprintf(stderr, "bitTwiddle is not bijective at y=%d\n", y);
            exit(1);
        }
    }
    inverseTwiddle = 1;
}

// brute force map byte
static uint8_t reverseMap(uint32_t w) {
    for (int b = 0; b < 256; ++b) {
        if (hash((uint8_t)b) == w) return (uint8_t)b;
    }
    fprintf(stderr, "ERROR: No byte maps to 0x%08x - aborting\n", w);
    exit(1);
}

// LFSR
static uint64_t rng;
static uint8_t lfsr(void) {
    uint64_t s = rng;
    uint64_t fb = ((s >> 63) ^ (s >> 62) ^ (s >> 60) ^ (s >> 59)) & 1ULL;
    s = ((s << 1) | fb) & MASK64;
    rng = s;
    return (uint8_t)(s & 0xFFu);
}

// Replicate the fisher-yates shuffle
static void shuffle(uint8_t js[USERNAME_LEN]) {
    for (int i = USERNAME_LEN - 1; i >= 0; --i) {
        uint8_t r = lfsr();
        js[i] = (uint8_t)(r % (i + 1));
    }
}

```

```

    }
}

// Reverse Fisher-yates
static void unshuffle(uint8_t a[USERNAME_LEN], const uint8_t js[USERNAME_LEN]) {
    for (int i = 0; i < USERNAME_LEN; ++i) {
        int j = js[i];
        uint8_t tmp = a[i];
        a[i] = a[j];
        a[j] = tmp;
    }
}

int main(void) {
    // build inverse of twiddling
    if (!inverseTwiddle) inverseByte();

    // inverse map byte
    uint8_t inversedMap[USERNAME_LEN];
    for (int i = 0; i < USERNAME_LEN; ++i) {
        inversedMap[i] = reverseMap(USERNAME[i]);
    }

    // set seed according to exe
    rng = stateSeed;

    // generate the same seed from Fisher-yates
    uint8_t js[USERNAME_LEN];
    shuffle(js);

    // unshuffle Fisher-yates
    uint8_t transformed[USERNAME_LEN];
    memcpy(transformed, inversedMap, USERNAME_LEN);
    unshuffle(transformed, js);

    // Bit twiddling
    uint8_t original[USERNAME_LEN + 1];
    for (int i = 0; i < USERNAME_LEN; ++i) {
        original[i] = inversePerByte[transformed[i]];
    }
    original[USERNAME_LEN] = '\0';

    printf("Recovered %d bytes:\n", USERNAME_LEN);

```

111

```
Recovered 34 bytes:  
Hex: 42 4f 48 32 35 7b 68 30 77 5f 77 34 35 5f 54 68 33 5f 70 34 74 48 5f 37 6f 5f 68 33 34 76 33 6e 3f 7d  
String: B0M25{h0w_w45_Th3_p4tH_7o_h34v3n?}
```

```

The Angels are watching closely

What is thy name?
BOH25{h0w_w45_Th3_p4tH_7o_h34v3n?}

The Angels pause...

The Gates of Heaven opens for the worthy.

```

Flag = BOH25{h0w_w45_Th3_p4tH_7o_h34v3n?}